

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

[Полное наименование образовательной организации]

Кафедра [наименование кафедры]

ОТЧЁТ

по лабораторной работе №1 по дисциплине «Защита информации»

Протокол Диффи-Хеллмана

Выполнил: студент группы [группа]
[Фамилия И. О.]

Проверил: [должность]
[Фамилия И. О.]

[Город] — 2026

1 Цель работы

Цель работы — изучить и проанализировать работу протокола безопасного обмена ключами по незащищённому каналу связи Диффи-Хеллмана; написать программу, реализующую работу протокола.

Для этого разработана программа `dh.py` (Python 3, стандартная библиотека) с режимами случайной генерации, ручного ввода, самопроверки по таблице 1.1 и перебора ключа третьим лицом.

2 Теоретические сведения

Открытые параметры — простое p и порождающий элемент α группы Z_p^* : его порядок, наименьшее $k \geq 1$ с $\alpha^k \equiv 1 \pmod{p}$, равен $p-1$. Порождающих элементов ровно $\varphi(p-1)$.

Обмен (рисунок 4.1): Алиса выбирает секретный x , посылает $A = \alpha^x \bmod p$; Боб выбирает секретный y , вычисляет $K_B = A^y \bmod p$, посылает $B = \alpha^y \bmod p$; Алиса вычисляет $K_A = B^x \bmod p$. Ключи совпадают:

$$K_A = (\alpha^y)^x \equiv \alpha^{xy} \equiv (\alpha^x)^y = K_B \pmod{p}.$$

По малой теореме Ферма $\alpha^{p-1} \equiv 1 \pmod{p}$, откуда редукция показателя по модулю $p-1$:

$$\alpha^x \equiv \alpha^{x \bmod (p-1)} \pmod{p}.$$

Секрет может превышать $p-1$ (строка 1 таблицы 1.1: $60 \bmod 28 = 4$, $K = 23$). Вычислять α^x целиком нельзя (при p , x порядка 10^4 — десятки тысяч цифр, в C и Java переполнение); `modpow` берёт остаток после каждого умножения, промежуточные значения не превышают $(p-1)^2$.

Стойкость сводится к задаче дискретного логарифмирования (по p , α , A найти x), при $p < 10000$ решаемой перебором не более $p-1$ степеней α за доли миллисекунды (таблица 6.5). Аутентификации сторон нет: активный противник подменяет A и B своим $\alpha^z \bmod p$ и разделяет ключ с каждой стороной — атака «человек посередине» (рисунок 7.1).

3 Постановка задачи

Задание методички: «Разработать программу, состоящую из частей Боба и Алисы (А и В). Один из участников генерирует ключ (< 10000) случайным образом, а затем организует передачу другому согласно протоколу Диффи-Хеллмана. В результате оба участника получают общий секретный ключ К. Числовые значения должны генерироваться случайным образом или вводиться с клавиатуры по желанию пользователя».

Семь требований: T1 — части Алисы и Боба с секретами x и y ; T2 — случайная генерация при ограничении « < 10000 »; T3 — по каналу проходят только p , α , $\alpha^x \bmod p$, $\alpha^y \bmod p$; T4 — $K_A = K_B$, о совпадении сообщается; T5 — ввод с клавиатуры; T6 — для 10 строк таблицы 1.1 ключ равен табличному К; T7 — ответы на контрольные вопросы. T1 подтверждает рисунок 5.1, T2–T6 — таблицы 6.1–6.5 и рисунок 6.1, T7 — раздел 8.

Допущения (`P_MIN`, `P_MAX`, `read_int`): 1) ограничение « < 10000 » отнесено к $p \in [5, 9999]$, откуда α , A , B , $K < 10000$; 2) инициатор (Алиса) генерирует p , α , x и передаёт p , α , $A = \alpha^x \bmod p$, но не К; 3) Алиса и Боб — объекты одного процесса, канал моделирует класс `Channel`, журналирующий сообщения; 4) α — порождающий элемент (`is_generator`): таких $\phi(p-1)$, для $p = 29$ — 12 из 28; 5) x , y не ограничены сверху, поскольку $\alpha^x \equiv \alpha^{x \bmod (p-1)} \pmod p$, а шесть строк таблицы 1.1 содержат x или $y \geq p-1$; 6) ручной ввод проверяется (целое; простое p из $[5, 9999]$; порождающее α из $[2, p-2]$; x , $y \geq 1$), иначе запрос повторяется.

4 Описание алгоритма

Алиса генерирует простое p , порождающий элемент α группы Z_p^* и секрет x , Боб — секрет y . По открытому каналу идут только p , α , $A = \alpha^x \bmod p$, $B = \alpha^y \bmod p$ (рисунок 4.1); секреты не передаются, ключ $K = B^x \bmod p = A^y \bmod p$ стороны вычисляют независимо.

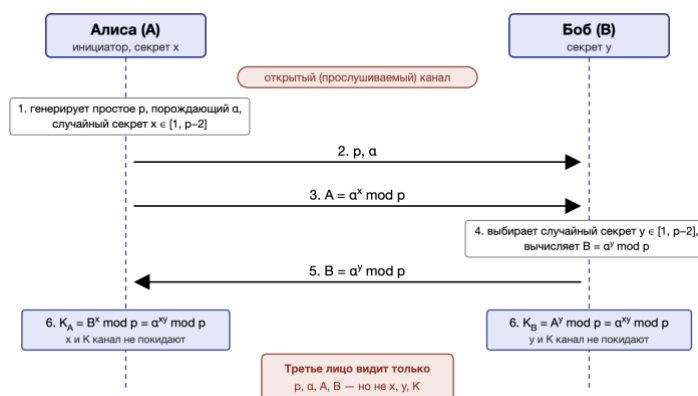


Рисунок 4.1 — Схема протокола Диффи-Хеллмана: что передаётся по открытому каналу

`gen_prime` повторяет случайный выбор из $[5, 9999]$, пока число не пройдёт `is_prime` — пробное деление на нечётные d до $\sqrt{n}$.

`element_order` находит порядок α — наименьшее k с $\alpha^k \equiv 1 \pmod{p}$ — прямым подсчётом за $O(p)$; `is_generator` требует $k = p - 1$. Порождающих около трети (12/28, 32/96, 3312/9972 для $p = 29, 97, 9973$), поэтому `gen_generator` тратит в среднем около трёх попыток.

Степени A, B, K вычисляет `modpow` — бинарный метод, остаток после каждого умножения:

```
modpow(base, exp, mod):
    если mod = 1: вернуть 0
    result ← 1; base ← base mod mod
    пока exp > 0:
        если exp нечётно: result ← result · base mod mod
        base ← base · base mod mod
        exp ← exp div 2
    вернуть result
```

Промежуточные произведения не превышают $(p - 1)^2 < 2^{31}$; вычислять a^e целиком через `double` нельзя: $22^{60} \bmod 29$ даёт 12 вместо 23.

Секрет — `random.randint(1, p - 2)`: $x = 0$ и $x = p - 1$ дают $A = K = 1$. При ручном вводе достаточно $x \geq 1$, так как $\alpha^x \equiv \alpha^{x \bmod (p-1)} \pmod{p}$.

`brute_force_dlog` моделирует наблюдателя: умножает на α по модулю p до получения A и возвращает $k = x \bmod (p - 1)$; этого достаточно: $K = B^k \bmod p$ (строка 1 таблицы 1.1: $k = 4 = 60 \bmod 28$, $K = 23$).

5 Описание программы

Программа на Python 3 (только стандартная библиотека) размещена в файле `dh.py` и запускается командой `python3 dh.py`.

Состав (рисунок 5.1): арифметика — `modpow`, `is_prime`, `element_order`, `is_generator`, `gen_prime`, `gen_generator`; участники и канал — `Channel`, `Participant`, `Alice`, `Bob`, `run_protocol`, `print_session`; проверка и атака — `self_test`, `brute_force_dlog`, `attack_demo`; интерфейс — `read_int`, `manual_mode`, `random_mode`, `main`.

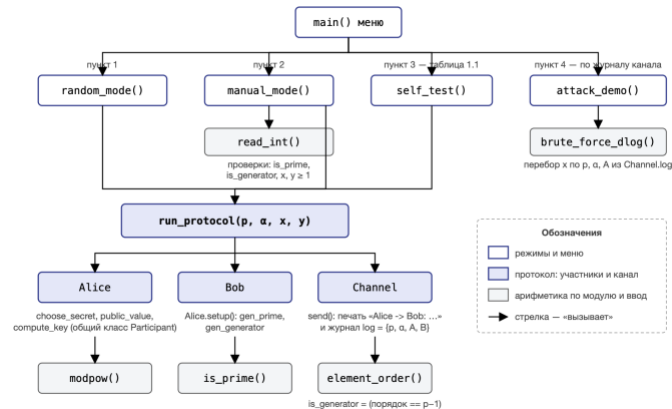


Рисунок 5.1 — Структура программы `dh.py`

Инициатор — Алиса: получает p и α , выбирает x , отправляет $A = \alpha^x \bmod p$; Боб отвечает $B = \alpha^y \bmod p$. Через `Channel.send` проходят ровно четыре передачи — p , α , A , B ; x , y , ключи в журнал `log` не попадают.

Меню: 1 — случайная генерация параметров (`random_mode`), 2 — ручной ввод (`manual_mode`), 3 — самопроверка по таблице 1.1 (`self_test`), 4 — перебор ключа третьим лицом по журналу последнего сеанса (`attack_demo`, x находит `brute_force_dlog`), 0 — выход; иначе — «Нет такого пункта меню.».

Сеанс выводит журнал канала, секреты сторон, вычисления K_A , K_B и итог сравнения ключей.

В режиме 2 параметры читает `read_int`: нечисловой ввод даёт «Нужно ввести целое число.», нарушение условия — сообщение параметра: « p должно быть простым числом из $[5, 9999]$.», « α должно лежать в $[2, p-2]$ и быть порождающим элементом Z_p^* (порядок α равен $p-1$).», « x должно быть ≥ 1 .» (аналогично y); запрос повторяется, x , y сверху не ограничены.

Завершение — пункт 0; `Ctrl+D` или `Ctrl+C` печатают «Выход.».

6 Результаты тестирования

Таблицы 6.1–6.5 получены прогоном функций программы из отдельного тестового сценария, рисунки 6.1–6.2 — экраны программы. Тб: десять строк таблицы 1.1 выполнены функцией `run_protocol` (пункт меню 3, `self_test`) — таблица 6.1.

Таблица 6.1 — Самопроверка по таблице 1.1 методички

№	p	a	x	y	A = $a^x \bmod p$	B = $a^y \bmod p$	К ожидаемый	К полученный	Совпало
1	29	2	60	82	16	22	23	23	да
2	53	2	20	77	24	26	42	42	да
3	73	5	37	47	68	42	31	31	да
4	31	3	31	51	3	15	15	15	да
5	41	6	31	98	13	33	8	8	да
6	19	2	20	73	4	2	4	4	да
7	71	7	30	89	32	56	45	45	да
8	109	6	7	14	24	31	84	84	да
9	131	2	88	15	108	18	52	52	да
10	97	5	58	68	44	4	6	6	да

Получено 10 совпадений из 10: $K_A = K_B$ равны ожидаемым К. Экран прогона — рисунок 6.2.

```

Лабораторная работа №1. Протокол Диффи–Хеллмана
1 – случайная генерация параметров
2 – ручной ввод параметров
3 – самопроверка по таблице 1.1
4 – перебор ключа третьим лицом (по журналу последнего сеанса)
0 – выход
Выбор: 3
№  р  а  x  y  А  В  К  ожд.  К  получ.  результат
1  29  2  60  82  16  22  23  23  PASS
2  53  2  20  77  24  26  42  42  PASS
3  73  5  37  47  68  42  31  31  PASS
4  31  3  31  51  3  15  15  15  PASS
5  41  6  31  98  13  33  8  8  PASS
6  19  2  20  73  4  2  4  4  PASS
7  71  7  30  89  32  56  45  45  PASS
8  109 6  7  14  24  31  84  84  PASS
9  131 2  88  15  108 18  52  52  PASS
10 97 5  58  68  44  4  6  6  PASS
Итог: 10/10 строк совпали с таблицей 1.1

```

Рисунок 6.2 — Самопроверка по таблице 1.1: 10/10 PASS

Все строки PASS, итог 10/10. Поскольку ключи совпадают при любом a и составном p , простота p и порядок a проверены отдельно на парах таблицы 1.1 и контрпримерах (таблица 6.2).

Таблица 6.2 — Контрпримеры к проверке простоты p и порождаемости a (все 10 пар из таблицы 1.1 проверку прошли: p простое, порядок $a = p - 1$)

№	р	р простое	α	порядок α	р - 1	α — порождающий
11	29	да	4	14	28	нет
12	7	да	2	3	6	нет
13	7	да	3	6	6	да
14	15	нет	2	—	14	— (р составное)

Все десять пар прошли. При $p = 29$ $\alpha = 4$ имеет порядок 14 вместо 28, при $p = 7$ порядки $\alpha = 2$ и $\alpha = 3$ равны 3 и 6, $p = 15$ составное. Редукция показателя по модулю $p - 1$ ключа не меняет: строка 1 — $60 \bmod 28 = 4$, $82 \bmod 28 = 26$, $K = 23$. Т2, Т4: пять сеансов `run_protocol()` без аргументов (пункт 1) — таблица 6.3.

Таблица 6.3 — Пять сеансов в случайном режиме

№	р	α	х	у	А	В	K_A	K_B	$K_A = K_B$	р простое, α пород.
1	1327	732	771	1052	59	1115	241	241	да	да
2	7499	5768	5869	4731	1372	4272	2006	2006	да	да
3	7559	3460	743	3267	2333	7126	1293	1293	да	да
4	9601	9086	8167	1908	1972	4784	8830	8830	да	да
5	7489	3847	5854	4470	6222	2033	3864	3864	да	да

Во всех сеансах $K_A = K_B$; p различны, простые, меньше 10000, α — порождающие. Т5: реакция `manual_mode` (пункт 2) на шесть вариантов ввода — таблица 6.4.

Таблица 6.4 — Реакция программы на ручной ввод

№	Ввод	Ожидаемая реакция	Фактическая реакция
1	$p = 15$ (составное)	отказ, повторный запрос p	сообщение « p должно быть простым числом...», повторный запрос
2	$p = 29$, $\alpha = 4$ (порядок 14, не порождающий)	отказ, повторный запрос α	сообщение « α должно лежать в $[2, p-2]$ и быть порождающим...», повторный запрос
3	$p = 29$, $\alpha = 1$ (вне диапазона $[2, p-2]$)	отказ, повторный запрос α	сообщение « α должно лежать в $[2, p-2]$ и быть порождающим...», повторный запрос
4	$x = 0$	отказ, повторный запрос x	сообщение « x должно быть ≥ 1 ...», повторный запрос
5	$x = \text{«abc»}$ (нечисловой ввод)	сообщение, повторный запрос x	сообщение «Нужно ввести целое число...», повторный запрос
6	$p = 29$, $\alpha = 2$, $x = 60$, $y = 82$ ($x > p - 1$)	принято, $K = 23$	принято, $K = 23$

Составное $p = 15$, $\alpha = 4$, $\alpha = 1$, $x = 0$ и строка `abc` отвергнуты с повторным запросом; $x = 60$, $y = 82$ ($p = 29$) приняты, хотя больше $p - 1$, $K = 23$. Рисунок 6.1: ручной ввод строки 1 и перебор ключа по журналу канала (пункт 4).

```
Лабораторная работа №1. Протокол Диффи-Хеллмана
1 – случайная генерация параметров
2 – ручной ввод параметров
3 – самопроверка по таблице 1.1
4 – перебор ключа третьим лицом (по журналу последнего сеанса)
0 – выход
Выбор: 2
Ручной ввод параметров.
p (простое, 5 <= p <= 9999): 29
alpha (порождающий элемент, 2 <= alpha <= p-2): 2
x (секрет Алисы, x >= 1): 60
y (секрет Боба, y >= 1): 82
Обмен по открытому каналу:
Alice -> Bob: p = 29
Alice -> Bob: alpha = 2
Alice -> Bob: A = alpha^x mod p = 16
Bob -> Alice: B = alpha^y mod p = 22
секрет Алисы x = 60, секрет Боба y = 82 (по каналу не передаются)
Алиса: K = B^x mod p = 22^60 mod 29 = 23
Боб: K = A^y mod p = 16^82 mod 29 = 23
Ключи совпали: общий сеансовый ключ K = 23

Выбор: 4
Перехвачено из канала: p = 29, alpha = 2, A = 16, B = 22
Перебор: x = 4 (по модулю p-1) найден за 4 итераций, 0.001 мс
Восстановленный ключ K = B^x mod p = 23
```

Рисунок 6.1 — Ручной ввод строки 1 таблицы 1.1 и перебор ключа по журналу канала

По перехваченным p , α , $A = 16$, $B = 22$ найден $x = 4$ за 4 итерации, $K = 23$. Контрольный вопрос 2: `brute_force_dlog` ищет наименьший k с $\alpha^k \equiv A \pmod{p}$, затем $K = B^k \pmod{p}$; $p = 29, 997, 9973$ — таблица 6.5.

Таблица 6.5 — Восстановление ключа третьим лицом перебором дискретного логарифма

p	α	x	A	найденный x	время, мс	K	K восст.	Совпало
29	2	60	16	4 (= 60 mod 28)	0.005	23	23	да
997	123	328	88	328	0.014	778	778	да
9973	8387	1682	4610	1682	0.065	2043	2043	да

Ключ восстановлен за 4, 328, 1682 итерации (0.005–0.065 мс); для $p = 29$ найдено $x = 4 = 60 \pmod{28}$. Т1–Т7 подтверждены таблицами 6.1–6.5, рисунками 6.1–6.2, разделом 8.

7 Анализ результатов

Все 10 строк таблицы 1.1 воспроизведены (таблица 6.1, рисунок 6.2), пять случайных сеансов (таблица 6.3) дали $K_A = K_B$.

Равенство ключей не доказывает корректность параметров: тождество верно при любом α и составном p , поэтому простота p и порождаемость α проверены отдельно (таблица 6.2). Порождающих в Z_p^* 33–43%: 12/28 ($p = 29$), 32/96 и 3312/9972 ($p = 97, 9973$), поэтому случайное α требует проверки. При $p = 29$, $\alpha = 4$ порядок 14: ключи совпадают, но перебор сокращается до 14 итераций; такое α отвергается (таблица 6.4).

При $p < 10000$ ключ восстанавливается перебором за доли миллисекунды (таблица 6.5: 0,065 мс при $p = 9973$). Перебор находит $x \bmod (p - 1)$, чего по теореме Ферма достаточно: для строки 1 найдено $4 = 60 \bmod 28$, $K = 23$ (рисунок 6.1). Ограничение « < 10000 » — учебное; реальный протокол требует p от 2048 бит.

Главная уязвимость — отсутствие аутентификации: активный противник подменяет A и B на $\alpha^z \bmod p$, получая ключ A^z с Алисой и B^z с Бобом, а стороны считают обмен успешным (рисунок 7.1). Открытые значения аутентифицируют подписями (STS), сертификатами (TLS), отпечатками ключей (SSH).

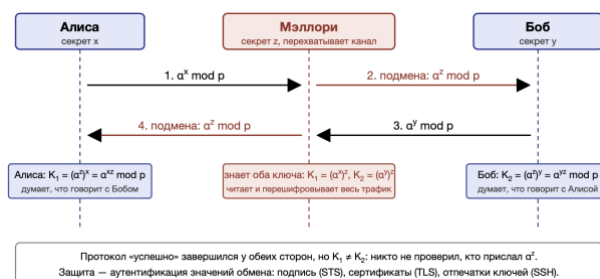


Рисунок 7.1 — Атака «человек посередине» на протокол без аутентификации

8 Ответы на контрольные вопросы

1. Для чего используется протокол Диффи-Хеллмана? Для чего нужен общий сеансовый ключ K ? Протокол вырабатывает общий секрет двух сторон по незащищённому каналу без заранее разделённого секрета: передаются только открытые p , α , A , B , а $K = \alpha^{xy} \bmod p$ стороны вычисляют сами. K служит ключом быстрого симметричного шифра для защиты последующего обмена; «сеансовый» — действующий в одном сеансе: новые x , y дают новый ключ (таблица 6.3).

2. Может ли третье лицо, прослушивая канал связи, получить общий сеансовый ключ K ? По каналу проходят только p , α , A , B ; получение K сводится к дискретному логарифмированию, при p длиной 2048 бит практически неосуществимому, поэтому ответ — нет. При учебных параметрах ответ обратный: `brute_force_dlog` перебирает k до $\alpha^k \equiv A \pmod{p}$; по таблице 6.5 для $p = 29$ показатель $x \bmod 28 = 4$ найден за 4 итерации и 0,005 мс, $K = 23$ (рисунок 6.1), при $p < 10000$ ключ вскрывается за доли миллисекунды. Стойкость требует $p = 2q + 1$ длиной не менее 2048 бит, порождающего α и случайных x , y .

3. В чём заключается основная уязвимость протокола Диффи-Хеллмана? Чем он, как правило, дополняется? Основная уязвимость — отсутствие аутентификации сторон, откуда атака «человек посередине» (рисунок 7.1): Мэллори подменяет A и B на $\alpha^z \bmod p$, получает с Алисой $K_1 = \alpha^{xz} \bmod p$, с Бобом — $K_2 = \alpha^{yz} \bmod p$ и перешифровывает сообщения; у сторон протокол завершается без ошибок. Поэтому протокол дополняется аутентификацией передаваемых значений: подписями открытых значений (STS), сертификатами (TLS), сравнением отпечатков ключей (SSH), протоколами на основе пароля (PAKE).

9 Выводы

Разработана программа `dh.py` со сторонами Алисы и Боба; канал передаёт только p , α , A , B (рисунок 6.1). Самопроверка пройдена для 10/10 строк таблицы 1.1 (таблица 6.1, рисунок 6.2); пять случайных сеансов дали простые $p < 10000$, порождающие α и равные ключи (таблица 6.3); ошибочный ввод отклоняется (таблица 6.4); ответы на контрольные вопросы — раздел 8; требования T1–T7 выполнены.

Показатели $x, y \geq p - 1$ допустимы по малой теореме Ферма; равенство $K_A = K_B$ не гарантирует порождаемости α (таблица 6.2); при $p < 10000$ секрет восстанавливается перебором (таблица 6.5), а без аутентификации возможна атака «человек посередине» (рисунок 7.1).

Приложение А. Листинг программы

Файл `dh.py`, Python 3, только стандартная библиотека. Запуск:

`python3 dh.py`.

```
#!/usr/bin/env python3
"""Лабораторная работа №1. Протокол Диффи-Хеллмана.

Программа состоит из частей Алисы и Боба. Алиса (инициатор) генерирует
открытые параметры  $p$  и  $\alpha$  и свой секретный показатель  $x$ , передаёт  $p$ ,  $\alpha$  и
 $A = \alpha^x \bmod p$  Бобу по открытому каналу. Боб выбирает секретный  $y$ , отвечает
 $B = \alpha^y \bmod p$ . Обе стороны вычисляют общий ключ  $K$ :
 $K_A = B^x \bmod p$ ,  $K_B = A^y \bmod p$ ,  $K_A == K_B == \alpha^{(xy)} \bmod p$ .

Режимы: случайная генерация параметров, ручной ввод с клавиатуры,
самопроверка по таблице 1.1 методички, перебор ключа третьим лицом
по журналу канала (почему учебное  $p < 10000$  не даёт стойкости).

Только стандартная библиотека. Запуск: python3 dh.py
"""

import random
import time

# --- Ограничения задания -----
# «Ключ (< 10000)» трактуем как ограничение на модуль  $p$ ; тогда все значения,
# идущие по каналу ( $p$ ,  $\alpha$ ,  $A$ ,  $B$ ), и сам ключ  $K$  тоже меньше 10000.
P_MIN = 5          # при  $p = 2, 3$  нет ни одного  $\alpha$  в диапазоне  $[2, p-2]$ 
P_MAX = 9999

# Таблица 1.1 методички — тест самопроверки: ( $p$ ,  $\alpha$ ,  $x$ ,  $y$ ,  $K$ )
TABLE_1_1 = [
    (29, 2, 60, 82, 23),
    (53, 2, 20, 77, 42),
    (73, 5, 37, 47, 31),
    (31, 3, 31, 51, 15),
    (41, 6, 31, 98, 8),
    (19, 2, 20, 73, 4),
    (71, 7, 30, 89, 45),
    (109, 6, 7, 14, 84),
    (131, 2, 88, 15, 52),
    (97, 5, 58, 68, 6),
]

# --- Арифметика по модулю -----
def modpow(base, exp, mod):
    """Быстрое возведение в степень по модулю (бинарный метод).

    Остаток берётся после каждого умножения, поэтому промежуточные значения
    не превышают  $(\text{mod}-1)^2$ . Считать  $\text{base}^{**\text{exp}}$  целиком не стоит: при  $p \sim 10^4$ 
    и  $x \sim 10^4$  это число из десятков тысяч цифр (в Java/C — переполнение,
    через double — потеря точности). Эквивалентно встроенному pow(base, exp, mod).
    """
    if mod == 1:
        return 0
    result = 1
    base %= mod
    while exp > 0:
        if exp & 1:
            # текущий бит показателя равен 1
            result = result * base % mod
        base = base * base % mod
        #  $\text{base}^{(2^i)}$ 
        exp = exp >> 1
```

```

        exp >= 1
    return result

def is_prime(n):
    """Проверка простоты пробным делением до sqrt(n). Для n < 10000 хватает."""
    if n < 2:
        return False
    if n % 2 == 0:
        return n == 2
    d = 3
    while d * d <= n:
        if n % d == 0:
            return False
        d += 2
    return True

def element_order(a, p):
    """Порядок элемента a в группе Z_p*: наименьшее k >= 1 с a^k ≡ 1 (mod p).

    Лобовой подсчёт: умножаем на a, пока не получим 1. O(p) шагов,
    для p < 10000 это доли миллисекунды. Определено только для простого p:
    при составном p множество {1..p-1} не является группой по умножению.
    """
    if not is_prime(p):
        raise ValueError("порядок элемента определён только для простого p")
    a %= p
    if a == 0:
        raise ValueError("a должно быть ненулевым по модулю p")
    k, value = 1, a
    while value != 1:
        value = value * a % p
        k += 1
    return k

def is_generator(a, p):
    """α — порождающий элемент Z_p*, если его порядок равен p-1,
    т.е. степени α^1..α^(p-1) пробегает все числа 1..p-1."""
    return 2 <= a <= p - 2 and element_order(a, p) == p - 1

def gen_prime(lo=P_MIN, hi=P_MAX):
    """Случайное простое из [lo, hi]."""
    while True:
        n = random.randint(lo, hi)
        if is_prime(n):
            return n

def gen_generator(p):
    """Случайный порождающий элемент Z_p*. Генераторов ровно φ(p-1) штук
    (в среднем около трети от p-1), поэтому нужно несколько попыток."""
    while True:
        a = random.randint(2, p - 2)
        if is_generator(a, p):
            return a

# --- Участники протокола и открытый канал -----
class Channel:
    """Открытый (прослушиваемый) канал: всё, что через него проходит,
```

```

печатается и записывается в журнал – это ровно то, что видит третье лицо."""

def __init__(self, verbose=True):
    self.verbose = verbose
    self.log = {} # метка -> значение, в порядке передачи

def send(self, sender, receiver, label, value):
    self.log[label] = value
    if self.verbose:
        print(f" {sender} -> {receiver}: {label} = {value}")
    return value

class Participant:
    """Общая часть Алисы и Боба: секретный показатель и две операции протокола."""

    def __init__(self, name):
        self.name = name
        self.secret = None # x у Алисы, y у Боба; канал не покидает
        self.key = None # общий ключ K после завершения протокола

    def choose_secret(self, p, value=None):
        self.secret = value if value is not None else random.randint(1, p - 2)
        return self.secret

    def public_value(self, alpha, p):
        """ $\alpha^{\text{secret}} \bmod p$  – единственное, что участник отправляет в канал."""
        return modpow(alpha, self.secret, p)

    def compute_key(self, other_public, p):
        """ $K = (\alpha^{\text{other}})^{\text{secret}} \bmod p$ ."""
        self.key = modpow(other_public, self.secret, p)
        return self.key

class Alice(Participant):
    """Инициатор: генерирует p,  $\alpha$  (если не заданы) и начинает обмен."""

    def __init__(self):
        super().__init__("Alice")

    def setup(self, p=None, alpha=None):
        p = p if p is not None else gen_prime()
        alpha = alpha if alpha is not None else gen_generator(p)
        return p, alpha

class Bob(Participant):
    def __init__(self):
        super().__init__("Bob")

def run_protocol(p=None, alpha=None, x=None, y=None, verbose=True):
    """Полный сеанс протокола. Возвращает (p,  $\alpha$ , x, y, A, B, K_A, K_B, канал)."""
    alice, bob = Alice(), Bob()
    channel = Channel(verbose)

    p, alpha = alice.setup(p, alpha)
    channel.send("Alice", "Bob", "p", p)
    channel.send("Alice", "Bob", "alpha", alpha)

```

```

x = alice.choose_secret(p, x)
A = channel.send("Alice", "Bob", "A = alpha^x mod p", alice.public_value(alpha,
p))

y = bob.choose_secret(p, y)
B = channel.send("Bob", "Alice", "B = alpha^y mod p", bob.public_value(alpha, p))

k_a = alice.compute_key(B, p)      # Алиса: K = B^x mod p
k_b = bob.compute_key(A, p)        # Боб: K = A^y mod p
return p, alpha, x, y, A, B, k_a, k_b, channel

def print_session(p, alpha, x, y, A, B, k_a, k_b):
    print(f" секрет Алисы x = {x}, секрет Боба y = {y} (по каналу не передаются)")
    print(f" Алиса: K = B^x mod p = {B}^{x} mod {p} = {k_a}")
    print(f" Боб: K = A^y mod p = {A}^{y} mod {p} = {k_b}")
    if k_a == k_b:
        print(f" Ключи совпали: общий сеансовый ключ K = {k_a}")
    else:
        print(" ОШИБКА: ключи различны")
    if A == 1 or B == 1:
        print(" Внимание: открытое значение равно 1 – секретный показатель кратен p-1, "
              "ключ вырожден (K = 1).")

# --- Самопроверка по таблице 1.1 -----
def self_test(verbose=True):
    """Прогон всех 10 строк таблицы 1.1. Возвращает число пройденных строк."""
    passed = 0
    if verbose:
        print(f" {'N':>2} {'p':>4} {'α':>2} {'x':>3} {'y':>3} {'A':>4} {'B':>4} {'K ожд.'>8} {'K получ.'>9} результат")
    for i, (p, alpha, x, y, k_expected) in enumerate(TABLE_1_1, 1):
        _, _, _, _, A, B, k_a, k_b, _ = run_protocol(p, alpha, x, y, verbose=False)
        ok = k_a == k_b == k_expected
        passed += ok
        if verbose:
            print(f" {i:>2} {p:>4} {alpha:>2} {x:>3} {y:>3} {A:>4} {B:>4} {k_expected:>8} {k_a:>9} {'PASS' if ok else 'FAIL'}")
    if verbose:
        print(f" Итог: {passed}/{len(TABLE_1_1)} строк совпали с таблицей 1.1")
    return passed

# --- Перебор дискретного логарифма (к контрольному вопросу 2) -----
def brute_force_dlog(p, alpha, target):
    """По перехваченным p, α, A = α^x mod p найти x перебором k = 1, 2, ...

    Возвращает наименьший k с α^k ≡ A (mod p): это x mod (p-1), а не сам x,
    но ключ по нему восстанавливается верно. Число итераций равно найденному k.
    Работает только потому, что p мало."""
    value = 1
    for k in range(1, p):
        value = value * alpha % p
        if value == target:
            return k
    return None

def attack_demo(channel):
    """Третье лицо знает только журнал канала: p, α, A, B – и ничего больше."""
    p, alpha = channel.log["p"], channel.log["alpha"]
    A, B = channel.log["A = alpha^x mod p"], channel.log["B = alpha^y mod p"]
    t0 = time.perf_counter()

```

```

x_found = brute_force_dlog(p, alpha, A)
elapsed_ms = (time.perf_counter() - t0) * 1000
k_found = modpow(B, x_found, p) if x_found is not None else None
print(f"  Перебавчено из канала: p = {p}, alpha = {alpha}, A = {A}, B = {B}")
print(f"  Перебор: x = {x_found} (по модулю p-1) найден за {x_found} итераций,
{elapsed_ms:.3f} мс")
print(f"  Восстановленный ключ K = B^x mod p = {k_found}")
return x_found, elapsed_ms, k_found

# --- Ввод с клавиатуры -----
def read_int(prompt, check=None, error="Недопустимое значение, повторите ввод."):
    """Читает целое число; при нечисловом вводе или нарушении check
печатает сообщение и спрашивает снова."""
    while True:
        raw = input(prompt).strip()
        try:
            value = int(raw)
        except ValueError:
            print("  Нужно ввести целое число.")
            continue
        if check is None or check(value):
            return value
        print(" ", error)

def manual_mode():
    """Ручной ввод p, alpha, x, y с проверками. Возвращает канал сеанса."""
    print("Ручной ввод параметров.")
    p = read_int(f"  p (простое, {P_MIN} <= p <= {P_MAX}): ",
                 lambda v: P_MIN <= v <= P_MAX and is_prime(v),
                 f"p должно быть простым числом из [{P_MIN}, {P_MAX}].")
    alpha = read_int("  alpha (порождающий элемент, 2 <= alpha <= p-2): ",
                     lambda v: is_generator(v, p),
                     "alpha должно лежать в [2, p-2] и быть порождающим элементом Z_p*")
    "
    " (порядок alpha равен p-1).")
    # Верхняя граница для x, y не проверяется: alpha^x ≡ alpha^(x mod (p-1)) (mod p),
    # в таблице 1.1 шесть строк из десяти имеют x или y >= p-1.
    x = read_int("  x (секрет Алисы, x >= 1): ", lambda v: v >= 1, "x должно быть >=
1.")
    y = read_int("  y (секрет Вобы, y >= 1): ", lambda v: v >= 1, "y должно быть >=
1.")
    print("Обмен по открытому каналу:")
    p, alpha, x, y, A, B, k_a, k_b, channel = run_protocol(p, alpha, x, y)
    print_session(p, alpha, x, y, A, B, k_a, k_b)
    return channel

def random_mode():
    """Случайная генерация всех параметров. Возвращает канал сеанса."""
    print("Случайная генерация параметров (Алиса — инициатор).")
    print("Обмен по открытому каналу:")
    p, alpha, x, y, A, B, k_a, k_b, channel = run_protocol()
    yes_no = lambda flag: "да" if flag else "нет"
    print(f"  свойства параметров: p простое — {yes_no(is_prime(p))}, p < 10000 —
{yes_no(p < 10000)}, "
          f"alpha порождающий — {yes_no(is_generator(alpha, p))}")
    print_session(p, alpha, x, y, A, B, k_a, k_b)
    return channel

def main():
    last = None # канал последнего сеанса — для атаки
    menu = ("\nЛабораторная работа №1. Протокол Диффи-Хеллмана\n"
            "  1 — случайная генерация параметров\n")

```

```

" 2 - ручной ввод параметров\n"
" 3 - самопроверка по таблице 1.1\n"
" 4 - перебор ключа третьим лицом (по журналу последнего сеанса)\n"
" 0 - выход")
try:
    while True:
        print(menu)
        choice = input("Выбор: ").strip()
        if choice == "1":
            last = random_mode()
        elif choice == "2":
            last = manual_mode()
        elif choice == "3":
            self_test()
        elif choice == "4":
            if last is None:
                last = random_mode()
            attack_demo(last)
        elif choice == "0":
            break
        else:
            print(" Нет такого пункта меню.")
    except (EOFError, KeyboardInterrupt):
        print("\nВыход.")

if __name__ == "__main__":
    main()

```